Requirements Specification

Project: RAG-Based Retrieval for Archival Systems
Version: 1.0
Date: 2026-06-01
Status: Draft

1. Introduction

1.1 Purpose

This document specifies the functional requirements for a Retrieval-Augmented Generation (RAG) system that provides natural-language access to institutional archival document collections. Users query a conversational interface; the system retrieves relevant document chunks and returns a grounded, cited response generated by a locally hosted LLM.

The system is built as a **microservices architecture**, with pipeline flows orchestrated by **Node-RED** and **Islandora** (via Drupal) as the archival repository.

1.2 Scope

In scope:

- Document ingestion (text extraction, chunking, embedding, indexing via Islandora)
- Natural language query processing and hybrid retrieval
- LLM-based response generation with source attribution
- Chat-based front-end (web extension and mobile)

Out of scope:

- OCR processing of scanned/image-based documents
- LLM fine-tuning
- Handwritten document processing
- Cloud inference APIs (all inference is on-premises)

1.3 Glossary

Term	Definition
RAG	Retrieval-Augmented Generation: combines LLM generation with external document retrieval
Chunk	A semantically bounded segment of a source document stored as a retrievable unit
Embedding	A dense numerical vector representing the semantic meaning of a text chunk
Dense Retrieval	Vector similarity search using embeddings
Sparse Retrieval	Keyword-based search (e.g., BM25)
Hybrid Retrieval	Combination of dense and sparse retrieval strategies
Reranker	A model that re-scores retrieved candidates by relevance to the query
Source Attribution	Tracing a generated response back to the specific source document(s) it was grounded in

Term	Definition
Contextual Chunk	A chunk enriched with structural metadata (section heading, document hierarchy, date)
Islandora	Open-source digital repository framework (via Drupal) used as the archival backend
Node-RED	Flow-based orchestration tool used to wire the microservices into ingestion and query pipelines
Microservice	An independently deployable service implementing one stage of the RAG pipeline

2. Functional Requirements

FR-01: Natural Language Query Interface

Description: The system shall accept natural language queries from users through a chat interface (web and mobile). Queries may be phrased as questions, keywords, or conversational prompts.

Priority: High
Testability: Submit a natural language question; verify the system returns a textual response within acceptable latency.

FR-02: Query Preprocessing

Description: The system shall preprocess each incoming query before retrieval:

- Language detection
- Query normalisation (lowercasing, punctuation handling)
- Keyword expansion where applicable

Priority: Medium
Testability: Submit queries in multiple languages and verify correct language detection; verify normalised forms are used in retrieval.

FR-03: Embedding Generation for Queries

Description: The system shall generate a dense vector embedding for each preprocessed query using a multilingual sentence transformer model.

Priority: High
Testability: Verify embedding dimensions match the vector database schema; semantically similar queries must produce geometrically close vectors.

FR-04: Hybrid Retrieval

Description: The system shall retrieve candidate chunks using a combination of:

- **Dense vector search**: semantic similarity via embeddings
- **Sparse keyword search**: BM25 or equivalent
- **Metadata / filter search**: document type, date range, collection tag

All three strategies shall be configurable and combinable.

Priority: High
Testability: Run identical queries with each strategy enabled/disabled; verify recall improves with hybrid mode active.

FR-05: Re-ranking of Retrieved Candidates

Description: The system shall apply a re-ranking model to the initial set of retrieved chunks to improve precision before context assembly.

Priority: Medium
Testability: Compare ranked vs. unranked output; top-k results after reranking must be more relevant than naive retrieval order.

FR-06: Context Assembly

Description: The system shall select and assemble the top-ranked chunks into a coherent context window for prompt construction, respecting the LLM's token limit. Chunks shall be ordered by relevance score.

Priority: High
Testability: Verify total assembled token count does not exceed the configured context budget.

FR-07: LLM Response Generation

Description: The system shall pass the assembled context and user query to a locally hosted open-weight LLM (via OLLAMA or vLLM) and return the generated response to the user. No external inference API may be used.

Priority: High
Testability: Verify responses are generated without external API calls; verify the response is grounded in the provided context.

FR-08: Source Attribution

Description: Every generated response shall include citations identifying the source document(s), section, and page (where available) from which the supporting evidence was retrieved.

Priority: High
Testability: For any factual claim in the response, the cited document must contain text that supports that claim.

FR-09: Structured JSON Chunk Storage

Description: The ingestion pipeline shall store document chunks as structured JSON objects preserving the document's semantic hierarchy with at minimum the following fields:

- `context_heading`: the section/chapter title the chunk belongs to
- `context_content`: the chunk's actual text
- `context_links`: references or related source identifiers
- `metadata`: document ID, date, language, collection tag

Priority: High
Testability: Inspect stored chunks in the vector DB; all required fields must be populated.

FR-10: Document Ingestion Pipeline

Description: The system shall support ingestion of new documents from the Islandora archival repository through an automated pipeline that:

1. Extracts text from structured digital documents (PDF, plain text, HTML)
2. Detects document language
3. Splits the document into contextual chunks using structure-aware chunking (not naive character-count splitting)
4. Generates embeddings for each chunk
5. Stores chunks with metadata in ChromaDB

Note: OCR processing of scanned image documents is out of scope.

Priority: High
Testability: Ingest a known PDF; verify all chunks appear in ChromaDB with correct fields and no mid-sentence chunk boundaries.

FR-11: Islandora Repository Integration

Description: The system shall integrate with Islandora (via Drupal) as the primary archival document store. New documents added to Islandora shall automatically trigger the ingestion pipeline.

Priority: High
Testability: Add a document to Islandora; verify the ingestion pipeline is triggered and the document appears in ChromaDB within the defined SLA.

FR-12: Microservices Architecture

Description: Each stage of the RAG pipeline (text extraction, chunking, embedding, retrieval, reranking, LLM generation) shall be implemented as an independently deployable microservice. Each service shall be independently replaceable without requiring changes to other services beyond its defined interface.

Priority: High
Testability: Stop or replace any single service (e.g., swap the embedding service) and verify the rest of the pipeline continues to operate against the defined interface contract.

FR-13: Node-RED Pipeline Orchestration

Description: The system shall use Node-RED to orchestrate the microservices into two flows:

- **Ingestion flow**: Islandora trigger → text extraction → language detection → chunking → embedding → indexing
- **Query flow**: query preprocessing → hybrid retrieval → reranking → context assembly → LLM generation → source attribution

Node-RED shall handle inter-service routing, triggering, and retry-on-failure.

Priority: High
Testability: Inspect the Node-RED flow; verify each pipeline stage maps to a node, and that a failed service invocation triggers the configured retry behaviour.

FR-14: Multilingual Support

Description: The system shall support retrieval and generation across multiple languages using multilingual sentence transformer models for embedding.

Priority: Medium
Testability: Ingest documents in German and English; submit queries in both languages; verify relevant results are returned for each.

FR-15: Session Context / Follow-Up Questions

Description: The system shall maintain conversational context within a session so that follow-up questions can reference prior turns without the user restating context.

Priority: Medium
Testability: Ask a follow-up question referencing a prior answer in the same session; verify the system resolves the reference correctly.

FR-16: Drupal Front-End (AI Chatblock)

Description: The system shall provide a chat-based front-end within **Drupal** (the same platform that hosts Islandora) using the **AI Chatblock** module (or its successor `ai_assistant_ui`). Responses shall be **streamed to the user via Server-Sent Events (SSE)** so replies appear progressively as they are generated.

Because AI Chatblock integrates through Drupal's `drupal/ai` AI-Assistant framework rather than accepting an arbitrary backend URL, the query microservice shall expose an **OpenAI-compatible, SSE-streaming endpoint** so it can be wired in via a Drupal AI provider pointed at the system's own endpoint.

Priority: Medium
Testability: Submit a query from the Drupal front-end; verify the response streams progressively (SSE), citations render correctly, and the backend receives the request via the query flow.

FR-20: Streaming Query API (OpenAI-Compatible)

Description: The query flow shall be exposed as an HTTP service (served via **Uvicorn / ASGI**) that:

- Accepts chat-style requests in an **OpenAI-compatible** request format
- Streams the generated response token-by-token via **Server-Sent Events (SSE)**
- Returns source-attribution metadata alongside the streamed content

This contract is the integration surface for the Drupal front-end (FR-16).

Priority: High
Testability: Send an OpenAI-format chat request to the endpoint; verify an SSE stream is returned and that an OpenAI-compatible client (or the Drupal AI provider) can consume it.

FR-21: Front-End Integration Spike

Description: Before committing to the AI Chatblock integration, the team shall validate the integration with a **dummy query endpoint/bot** implementing the agreed **dummy API contract** (OpenAI-compatible chat-completions + SSE). The spike shall confirm AI Chatblock can communicate with the dummy endpoint end-to-end.

Rationale: If AI Chatblock cannot integrate with the system's endpoint, the fallback is a **custom Drupal plugin/widget**, which widens project scope and must be flagged before commitment.

Priority: High
Testability: Configure AI Chatblock against the dummy endpoint; verify a streamed response is rendered in the Drupal chat widget without any custom plugin code.

FR-17: Vocabulary Mismatch Bridging

Description: The retrieval layer shall surface documents containing historical or domain-specific terminology when queried with modern equivalent phrasing (e.g., a query for "aviation" retrieves documents using "flying machines"). This shall be achieved through semantic dense retrieval without requiring user configuration.

Priority: High
Testability: Define a vocabulary-mismatch test set; verify recall@10 0.8 for paired modern/historical query equivalents.

FR-18: Document Removal from Index

Description: The system shall support removing a document from the retrieval index (deleting all its chunks from ChromaDB and the metadata index) so that deprecated or restricted content is no longer returned in responses.

Priority: Medium
Testability: Remove a document; verify no subsequent query returns content from that document.

FR-19: Ingestion Status Visibility

Description: The system shall expose per-document ingestion status to archivist/admin users: pending, processing, indexed, or failed. Failed ingestions shall include a human-readable error message.

Priority: Low
Testability: Submit a document; observe status transitions from pending → processing → indexed in the admin interface.

3. Constraints

ID	Constraint
C-01	OCR for scanned/image-based documents is out of scope; only machine-readable text documents are ingested.
C-02	LLM fine-tuning is out of scope; retrieval compensates for domain knowledge gaps.
C-03	All inference and document storage must remain on-premises; no public cloud APIs for document content.
C-04	Hardware target is a single GPU node with 24 GB VRAM (NVIDIA L4 or RTX 4090 equivalent).
C-05	Python services use `pyproject.toml` for dependency/build configuration and are served with Uvicorn (ASGI).
C-06	The Drupal front-end uses the AI Chatblock module; integration must work without a custom plugin, or scope expands (see FR-21).

ID	Constraint
C-07	The LLM / inference runtime runs on the university GPU server, accessed over SSH (e.g., an SSH tunnel to the model endpoint). The 24 GB GPU node (C-04) refers to this server.

4. Research Gaps (Future Scope)

The following are identified research gaps and are **not** part of v1.0:

- Evaluation benchmarks specific to archival retrieval tasks
- Handwritten and multimodal archival material processing
- Provenance-aware answer generation
- Privacy-preserving RAG
- Explainable retrieval interfaces
- Agentic archival research assistants (multi-step autonomous research)